

Smart Home Alarm System - Report

Martin Tomassi

1 Problem analysis

The system must process asynchronous messages from three independent sources: peripheral sensors, keypad inputs and timers. Because sensors and the keypad send messages without knowing the system's current state, all state logic must be centralized within the control unit. This requires state-dependent message processing, where the exact same message produces entirely different effects depending on whether the system is disarmed, armed, or in a delay state. Managing delay-based state transitions introduces the challenge of timer cancellation. During the exit delay, the system starts a countdown timer to go from disarmed to armed state: the control unit must be able to process an incoming valid pin message as a cancellation request before the timer completes the transition. When in the Armed state, the control unit must filter incoming sensor messages against the active arming mode (full or partial) to decide whether to initiate an entry delay or ignore the event. During this entry delay, the system must guarantee that receiving a valid pin message cancels the scheduled timeout, preventing an unwanted transition to the alarm state. As a final requirement, when in the alarm state, the control unit must remain responsive to keypad messages, so the user can silence the siren and disarm the system.

2 General strategy

The implementation follows the Actor model using Apache Pekko, organizing actors into a tree managed by **UserGuardian**. All domain logic and state transitions are centralized inside **ControlUnitActor**, while peripheral actors (**SensorActor**, **KeypadActor**, **SirenActor**) communicate exclusively with the **ControlUnitActor** and never directly with each other. All interactions, between actors, rely on asynchronous message passing, structured according to the responsibilities of each actor:

- **SensorActor** represents physical detection devices (motion or door/window sensors). When triggered, it sends a message to **ControlUnitActor** specifying which sensor and zone have detected activity.
- **KeypadActor** represents the numeric keypad. It forwards user actions (full or partial arming requests and PIN entries) directly to **ControlUnitActor**.
- **SirenActor** represents the alarm siren. It activates or deactivates the sound based on explicit messages from **ControlUnitActor**.
- **ControlUnitActor** serves as the central finite state machine operating through five states (*disarmed*, *exitDelay*, *armed*, *entryDelay*, and *alarm*), evaluating incoming messages from the peripherals and the timer in relation to the active monitoring zones.

State within **ControlUnitActor** is maintained immutably by passing contextual data as parameters between behaviours. Because an actor processes messages sequentially, sensor events, user commands and timer signals are evaluated atomically without concurrency issues. Delays rely on Pekko's **TimerScheduler** to send timeout messages, which are cancelled upon receiving a valid PIN. System startup and shutdown are managed by **UserGuardian**, which spawns actors, schedules scenario events and terminates the actor system upon completion.